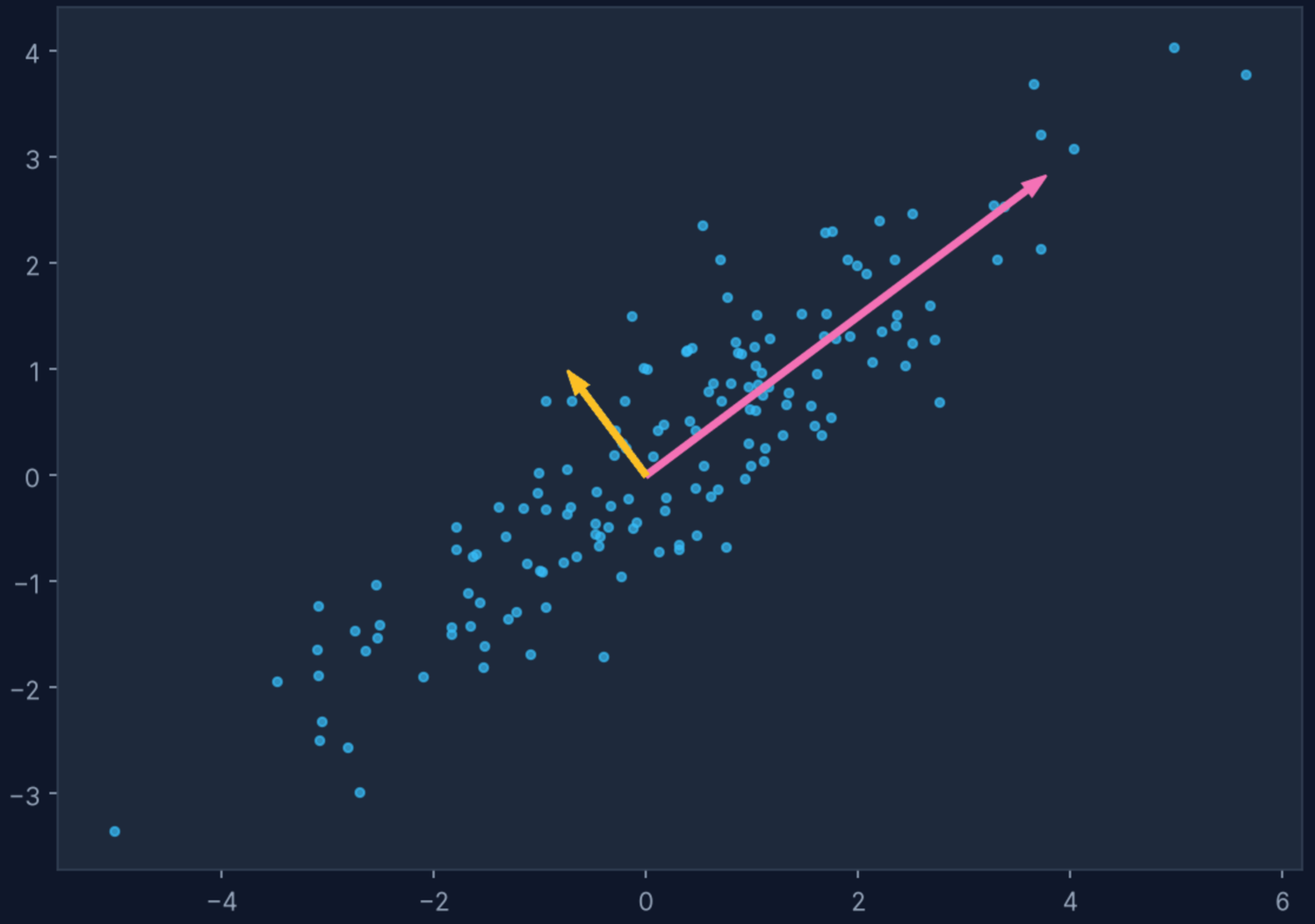
PCA

Principal Component Analysis via SVD

The problem

- Hundreds of correlated features
- Hard to visualise, slow to train
- Want fewer features, minimal information loss

Intuition: directions of spread



PC1: direction of most variance

PC2: next, orthogonal to PC1

Project onto the top k

Via SVD

$$X - \mu = U \Sigma V^\top$$

$$Z = (X - \mu)\, V_k, \qquad \mathrm{var}_i = \frac{\sigma_i^2}{n - 1}$$

- Rows of V^T are the components
- Explained variance ratio = σ_i^2 / $\Sigma\sigma^2$

PCA

```
class PCA:
    def __init__(self, n_components):
        self.n_components = n_components

    def fit(self, X):
        X = np.asarray(X, float)
        self.mean_ = X.mean(0)
        _, s, vt = np.linalg.svd(X - self.mean_, full_matrices=False)
        self.components_ = vt[:self.n_components]
        var = s ** 2 / (len(X) - 1)
        self.explained_variance_ratio_ = var[:self.n_components] / var.sum()
        return self

    def transform(self, X):
        return (np.asarray(X, float) - self.mean_) @ self.components_.T

    def inverse_transform(self, Z):
        return Z @ self.components_ + self.mean_
```

Complexity & trade-offs

- Thin SVD: $O(n \cdot d \cdot \min(n, d))$
- Scale features first if units differ
- Linear only; components can be hard to interpret
- Choose k to keep e.g. 95% variance

Recap

- Centre, SVD, keep top-k directions
- Components are orthogonal, ordered by variance
- Use for compression, visualisation, denoising